Comp 533 - Assignment 3: Distributed Map-Reduce

Date Assigned: Tuesday, February 11, 2025

Early Completion Date: Tuesday, Feb 25, 2025, 11:55 pm (+5% extra credit)

Completion Date: Friday, Feb 28, 2025, 11:55 pm

Late Date: Friday, Mar 07, 2025 (-10% late penalty)

Scary Late Date: Friday, Mar 21, 2024 (-30 % late penalty)

The assignment aims to give to practice in

- creating Java `Serializable` classes and interfaces
- creating Java `Remote` classes and interfaces
- handling `RemoteException`
- creating RMI `Registry` as part of a server process
- registering and looking up `Remote` instances
- implementing reduction in client processes
- implementing a distributed quit
- implementing Facebook mutual friends through MapReduce

In the previous assignment, you split a map among multiple slave *threads*. In this assignment, you will be splitting it amongst both slave *threads and* slave *processes*. For inter-process communication, you will use Java RMI (Remote Method Invocation). The main process from A2 will now become a server with which client processes can register. Each client process is assigned to a currently unassigned slave thread inside the server, and some slave threads may remain unassigned. An assigned slave thread forwards its work to its assigned client process. Thus, the client processes are the ones providing the services needed for this problem and the server process is the one using the services! For this assignment, the RMI Registry process should be part of the server process.

The class material covers the Java features needed for this project. Any tutorial on Java RMI should also work, though it may not cover in as much depth the underlying concepts. The `Serializable` interface referenced in the class material is in the package `java.io`. The other referenced interfaces and classes (e.g. `Remote`, `LocateRegistry`, `UnicastRemoteObject`) are in the package `java.rmi`. The code for the class RMI examples is in packages starting with `examples.rmi` and `examples.serializable` in the GitHub repository, GIPC, which includes all examples covered in class.

Graphically Illustrated Algorithm Overview

In this assignment, unlike the last two, the goal is less to implement an algorithm but more to create a distributed program out of our work from the last two assignments. The server's job is to run and facilitate the threads and MVC. The following figure illustrates the various steps in this distributed algorithm.

First, the server should create an RMI `Registry` and export a proxy of its server model to it (Fig. 1).

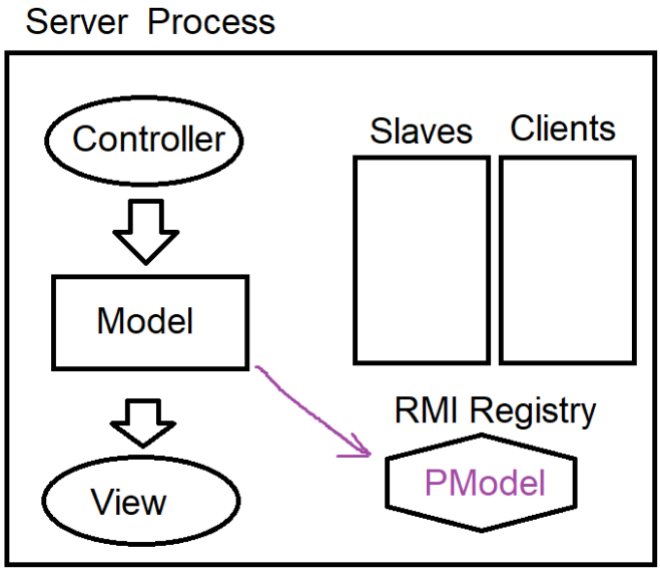


Figure 1. Initial State of Server: No slave or client references but Proxy for Model Registered

A user can set the number of slave threads and clients can join at any time after this initial step. In the next step, we assume that a user enters the number of threads before a client registers (Fig. 2).

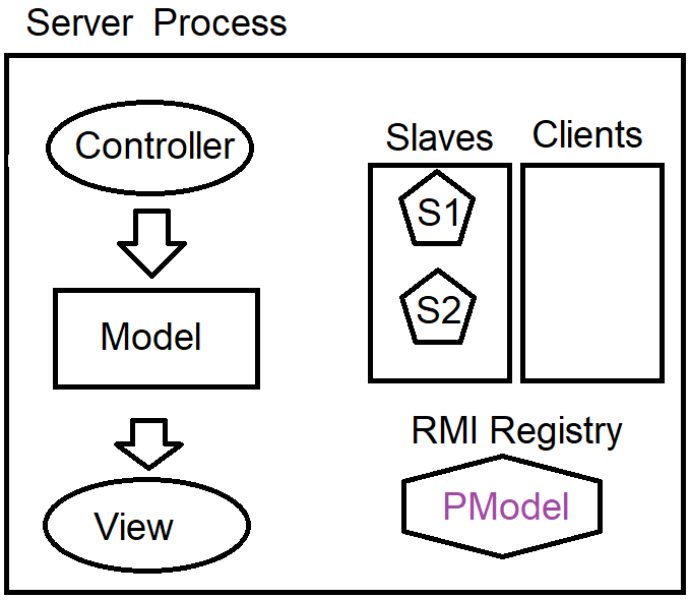


Figure 2. Server after local threads created: References to slaves now filled

The server will need to call methods in the client, which means the server needs a proxy of the client object on which the server's remote calls are made. We will call this the client proxy.

In the next step (Fig. 3), a client will acquire a proxy for the server's model through the RMI registry and register its own client proxy with the server by making a remote call. (**The client will not register its proxy with an RMI registry. An RMI registry lets you look up objects by name and contact them before they contact you first. A client has reason to look up a server by name, but a server learns about clients when they talk to it; it doesn't go looking for them.**)

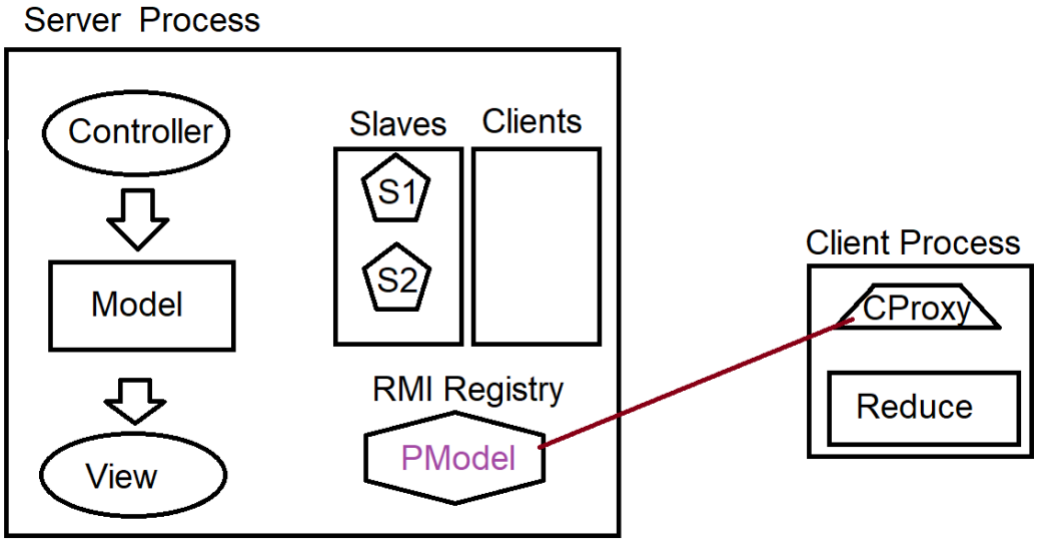


Figure 3. Client registering with the server: Client is sending its client proxy to the server via a remote method call.

To register the client, the server will enter the client proxy in a registry it keeps (which is **not** the same as the RMI Registry) and associate the client proxy with a slave that doesn't have a client. (Remember that you will also have to handle the case if a client registers before slaves are created.)

When a slave is associated with a client, it delegates its reduction task to the client. Thus a client is a worker to the slave thread, just as a slave thread is a worker to the master main thread.

Let us say that the Server then is given the following input: "a a b z z" and slave1's initial list is "a z z" and slave2's is "a b". Since slave1 has a reference to a client proxy it will use that reference to make the client execute the reduce step (Fig. 3). Slave2 meanwhile will execute the reduce step on its own as it has not been assigned a client proxy.

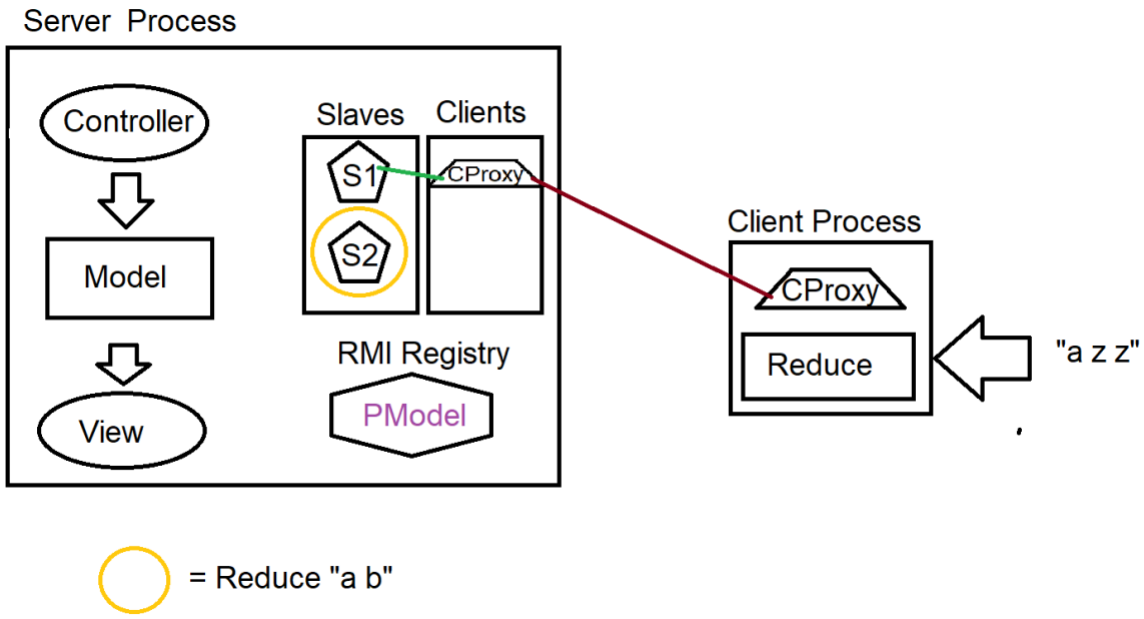


Figure 4. Server delegates work to client. Server has stored the client proxy and bound it to Slave 1. Slave 1 performs its reduction by making a remote method call to the client. Slave 2 does its reduction by itself.

Th e client's only job is to help handle the reduction steps. The rest of the algorithm is done on the server through the slave thread to which it is connected.

The goal of the assignment is to make this division of labor work.

New Configuration Methods (Regular Credit)

For LocalChecks regular credit, you need to implement the following additional methods in your configuration class:

```
// The model stored in the server will now have a remote interface to be
   used by the client to register its proxy.
// The model class will have to be changed to implement  this interface.
// If you change the name of this class, make sure you change the
   getModelClass
// method of the registry to reflect the name change.
Class getRemoteModelInterface();


// The client proxy also has a remote interface for remote callbacks
Class getRemoteClientObjectInterface();

// This is the class implementing the client interface above
Class getRemoteClientObjectClass();

// For each application (token count, int sum), we now have two main
   classes, one for the server and one for each client.
// 1. The main class of the server token counter.
```

```
Class getServerTokenCounter();

// 2. The main class of the server int summer.
Class getServerIntegerSummer();

// This is the main class of both the client token counter and the int
   summer, since the reduction step is the same for both, and the client
   performs only reduction
Class getClientTokenCounter();
```

Distributed Map-Reduce

Use RMI to convert the A2 solution to a distributed application with the following extensions:

1. *Single-process program → server:* A server process now implements the A2 code with additional features that allow it to support remote slave clients. The RMI `Registry` should be part of the server process. This means the server will call `createRegistry` and not `locateRegistry`, and you do not need to take any special steps to keep the registry alive, as the export by the server process will do so. In later assignments, the registry will be a separate process and you will have to call `locateRegistry` in the server, too.
2. *Client process*: In addition, you will start client processes. In Eclipse you can run multiple programs at the same time to create different processes. Which process's console interaction is shown in the console window depends on which process you select in the debug window shown in the debug perspective. All client processes will run the same client `main`.
3. *Single-process model → Server `Remote` model*. The model object in the standalone process now becomes a `Remote` object in the server, on which client processes can remotely invoke methods. Specifically, a client process's `main` method will call a method in the remote server model to register a *client `Remote` object*. The server model keeps track of all of the registered client objects as Fig. 4 shows.
4. *Slave-client match function:* Some or all of the client objects are assigned to some or all of the slave runnables. The server model keeps track, in `java.util.Stack` instances, a stack of all unassigned slave runnables and client objects. It implements a method to match the two stacks, which connects the last (most recent) items in the two stacks, if they both exist, to each other, and pops those items after the connection is made, repeatedly matching elements from the two stacks until one of them is empty. Thus, it is possible to have unmatched slaves (if the number of clients is less than the number of slaves) and it is also possible to have unmatched clients (if the number of slaves is less than the number of clients), but your server should avoid the condition that both a client and a slave thread are unmatched at the same time. This is the trickiest part of the assignment.
5. *Invoking the match function:* The match function is called (a) each time a new client is registered and (b) after the slave runnables (and associated threads) have

been created in response to user input. (Why in both situations; why not only (a) or only (b)?).

6. *Invoking client reduce callbacks:* If they have been assigned client objects, slave threads in the server (remotely) ask the client objects to do partial and final reduction of sub-lists. Otherwise, they perform local processing of the sub-lists to compute their results.
7. *Client reduce:* A client remote object provides a `reduce()` method that can be called remotely by the server runnable to which it has been assigned. This method accepts a list of **serializable** Key-Value objects, processes this list and returns a `Map` with the result. The client `reduce()` method simply calls the `reduce()` method of the reducer object retrieved from the client reducer factory. Both the server and client share the same reducer and reducer factory code. The trace methods `traceRemoteList()` and `traceRemoteResult()` should be called in the server and client at the start and end of the remote reduction, as shown in the trace below.
8. *Serializable Key-Value:* Your server slave will send a list of Key-Value Objects to the client, and receive a `Map<String, Integer>` back. The predefined `Lists`, `Maps`, `Integers`, and `Strings` are all `Serializable`. The type *you* previously defined for Key-Value Objects will **not** be `Serializable` by default. You must make the interface of this type a subtype of `Serializable` to allow its instances to be sent and received remotely.
9. *Client quit*: A client object also provides a method to `quit()`, which terminates the client.
10. *Terminating clients from server*: The `quit()` method of the server model calls the `quit()` method in all registered clients. This method (invoked by an RMI-created thread) must ensure that the client's main thread terminates. You should do this by having the client `main` call a `synchronized` method in the client object it registers that invokes `wait()` on the client object and invokes `System.exit()` when it unblocks from this call. The client `quit()` method should then call `notify()` on the client object (in a `synchronized` method) to unblock the main thread and enable execution of the `System.exit()` method. The `quit()` model method also calls `System.exit()` in the server to terminate all server threads.

Token Counting Server

The following trace excerpts illustrates the interaction above when the server and clients compute token counts:

```
Thread[main,5,main]:Controller:Please enter the number of threads
// A client registers before slave runnables are created
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI
Runtime]:Model:Register:Proxy[ClientKeyValueProcessor,RemoteObjectInvocationHa
ndler[UnicastRef [liveRef: [endpoint:[192.168.86.211:62685](remote),objID:[-
65e3a497:169446bf003:-7ffe, 44756118140031834351]]]]]
3
```

// Three slaves and runnables are created as before
```
Thread[Slave0,5,main]:Slave:dequeue:[]
Thread[Slave1,5,main]:Slave:dequeue:[]
Thread[Slave2,5,main]:Slave:dequeue:[]
```
// The most recent slave (2)'s runnable is assigned the remote client object
```
Thread[main,5,main]:Slave:Client
Assigned:Proxy[ClientKeyValueProcessor,RemoteObjectInvocationHandler[UnicastRe
f [liveRef: [endpoint:[192.168.86.211:62685](remote),objID:[-
65e3a497:169446bf003:-7ffe, 4475611814003183435]]]]]
…
```
// Again, the tokens to be counted
```
Thread[main,5,main]:Controller:Please enter quit or a line of tokens to be
processed separated by spaces
orange apple orange mango lychee lychee soursop lychee orange mango lychee
Thread[main,5,main]:Slave:notify
Thread[main,5,main]:Slave:notify
Thread[main,5,main]:Slave:notify
…
)
```
// Slave 2 dequeues its end of input
```
Thread[Slave2,5,main]:Slave:dequeue:(null,null)
```
//Slave 2 sends its list to its remote client
```
Thread[Slave2,5,main]:Slave:Remote List:[(orange,1), (lychee,1), (orange,1)]
```

// Slaves 0 and 1 dequeue their end of input
```
Thread[Slave0,5,main]:Slave:dequeue:(null,null)
Thread[Slave1,5,main]:Slave:dequeue:(null,null)
```
// Slaves 0 and 1 process their sub-lists locally
```
...
Thread[Slave1,5,main]:Reducer:reduce:[(apple,1), (lychee,1), (lychee,1),
(lychee,1)]:{apple=1, lychee=3}
Thread[Slave0,5,main]:Reducer:reduce:[(orange,1), (mango,1), (soursop,1),
(mango,1)]:{orange=1, soursop=1, mango=2}
```

// Slave 2 gets the result from the remote client
```
Thread[Slave2,5,main]:Slave:Remote Result:{orange=2, lychee=1}
```

// Slave 2 does the partitioning locally
```
Thread[Slave2,5,main]:Partitioner:Partition Assigned:orange:2:1:3
Thread[Slave2,5,main]:Partitioner:Partition Assigned:lychee:1:1:3
```
// Slave 2 sends its partition to the remote client
```
Thread[Slave2,5,main]:Slave:Remote List:[(soursop,1)]
```
// It receives the remote result and locally adds it to the final result
```
Thread[Slave2,5,main]:Slave:Remote Result:{soursop=1}
Thread[Slave2,5,main]:Model:Added to Final Map:{orange=3, lychee=4, apple=1,
soursop=1, mango=2}:{soursop=1}
```
// it adds the result locally

…
// Next round

```
Thread[main,5,main]:Controller:Please enter quit or a line of tokens to be
processed separated by spaces
```
// Before the user inputs, a new client registers with the server
```
Thread[RMI TCP Connection(4)-192.168.86.211,5,RMI
Runtime]:Model:Register:Proxy[ClientKeyValueProcessor,RemoteObjectInvocationHa
ndler[UnicastRef [liveRef:
[endpoint:[192.168.86.211:62703](remote),objID:[4c12ee2:169446c70b0:-7ffe,
7703970965893319657]]]]]
```
// Slave 1's runnable is assigned this new client
```
Thread[RMI TCP Connection(4)-192.168.86.211,5,RMI Runtime]:Slave:Client
Assigned:Proxy[ClientKeyValueProcessor,RemoteObjectInvocationHandler[UnicastRe
f [liveRef:
[endpoint:[192.168.86.211:62703](remote),objID:[4c12ee2:169446c70b0:-7ffe,
7703970965893319657]]]]]
```
// Repeat of the first input
```
orange apple orange mango lychee lychee soursop lychee orange mango lychee
```
…
// Slave 2 gets its end of input
```
Thread[Slave2,5,main]:Slave:dequeue:(null,null)
```
// Again it forwards its sub-list to the first remote client
```
Thread[Slave2,5,main]:Slave:Remote List:[(orange,1), (lychee,1), (orange,1)]

]
```
// Slave 0 gets its end of input
```
Thread[Slave0,5,main]:Slave:dequeue:(null,null)
```
// As before, Slave 0 processes its sub-list locally
...
// Slave 1 retrieves its end of input
```
Thread[Slave1,5,main]:Slave:dequeue:(null,null)
```
// This time it forwards its sub-list to its newly assigned (second) client
```
Thread[Slave1,5,main]:Slave:Remote List:[(apple,1), (lychee,1), (lychee,1),
(lychee,1)]
```

…
```
Thread[main,5,main]:Controller:Please enter quit or a line of tokens to be
processed separated by spaces
```
// The user types quit
```
Quit
```
// Server main sends interrupted exception to slave threads, calls the quit() method in client objects, and quits
```
java.lang.InterruptedExceptionThread[main,5,main]:Controller:Quitting
```
// Server slave threads receive the InterruptedException

```
        at java.base/java.lang.Object.wait(Native Method)
        at java.base/java.lang.Object.wait(Object.java:328)
        at
mapreduce.AnAbstractMasterSlave.synchronizedWait(AnAbstractMasterSlave.java:14
7)
        at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:64)
        at java.base/java.lang.Thread.run(Thread.java:844)
java.lang.InterruptedException
        at java.base/java.lang.Object.wait(Native Method)
```

```
	at java.base/java.lang.Object.wait(Object.java:328)
	at
mapreduce.AnAbstractMasterSlave.synchronizedWait(AnAbstractMasterSlave.java:14
7)
	at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:64)
	at java.base/java.lang.Thread.run(Thread.java:844)
java.lang.InterruptedException
```
// Server slave threads quit
```
Thread[Slave1,5,main]:Slave:Quitting
Thread[Slave2,5,main]:Slave:Quitting
	at java.base/java.lang.Object.wait(Native Method)
	at java.base/java.lang.Object.wait(Object.java:328)
	at
mapreduce.AnAbstractMasterSlave.synchronizedWait(AnAbstractMasterSlave.java:14
7)
	at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:64)
```

Token Counting Client 1

// Main thread registers its client object and waits on this object
```
Thread[main,5,main]:Client:wait
```
// Round 1: First User Input
// Client object gets remote list from Slave 2
```
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Remote
List:[(orange,1), (lychee,1), (orange,1)]
```
// Client object returns its partial counts to Slave 2 using the reducer's reduce
```
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI
Runtime]:Reducer:reduce:[(orange,1), (lychee,1), (orange,1)]:{orange=2,
lychee=1}

Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Remote
Result:{orange=2, lychee=1}
```
// Now it gets Slave2's partition and processes it
```
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Remote
List:[(soursop,1)]
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI
Runtime]:Reducer:reduce:[(soursop,1)]:{soursop=1}
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Remote
Result:{soursop=1}
```

// Round 2: Second User Input
A similar sequence occurs

...
// Quit method called in client by server
```
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Quitting
```
// Client object calls notify() on itself
```
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI
Runtime]:Client:notify
```
// Client main method exits()

```
Thread[main,5,main]:exit
```

Token Counting Client 2

// Joins after round 1

```
Thread[main,5,main]:Client:wait
```

// Receives round 2 sub-list

```
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Remote
List:[(apple,1), (lychee,1), (lychee,1), (lychee,1)]
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI
Runtime]:Reducer:reduce:[(apple,1), (lychee,1), (lychee,1),
(lychee,1)]:{apple=1, lychee=3}
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Remote
Result:{apple=1, lychee=3}
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI Runtime]:Client:Remote
List:[(orange,1), (mango,2), (orange,2), (lychee,1), (lychee,3)]
Thread[RMI TCP Connection(2)-192.168.86.211,5,RMI
Runtime]:Reducer:reduce:[(orange,1), (mango,2), (orange,2), (lychee,1),
(lychee,3)]:{orange=3, lychee=4, mango=2}
```

// Quit

```
Thread[RMI TCP Connection(1)-192.168.86.211,5,RMI Runtime]:Client:Quitting
Thread[RMI TCP Connection(1)-192.168.86.211,5,RMI Runtime]:Client:notify
Thread[main,5,main]:exit
```

Implement the client object `toString()` method to return the `CLIENT` constant in the tracer class:

```
// value returned by toString() method of the client proxy
public static final String CLIENT = "Client";
```

Distributed Summation

For distributed summation, all you need to do is create a different server launcher that creates the int summing model. The clients will remain the same.

Facebook Mutual Friends Map Reduce

For this partt you will need to implement Facebook's mutual friend map reduce algorithm into your assignment.

The problem can be stated as follows. Given the list of friends each user has, find the list of common friends each pair of friends has.

Let's say there are four people A, B, C, and D. A, B, and C are all friends with each other and D is only friends with C.

This example is represented by the input:

a,b,c b,a,c c,a,b,d d,c

The program should indicate for each pair of users in this four-person domain a list of mutual friends:

a_b = [c], a_c = [b], b_c = [a], a_d = [], b_d = [], c_d = []

Here our mapper will take the list of friends and return *n* number of key value pairs where *n* is the number of friends a specific person has. The key is a string that is a concatenation of a person and one of their friends (in alphabetical order by name), separated by _, and the value is the user's friend list. For example, the mappings we will get from our scenario are:

From user A:

A_B: [B,C]

A_C: [B,C]

From user B:

A_B: [A,C]

B_C: [A,C]

From user C:

A_C: [A,B,D]

B_C: [A,B,D]

C_D: [A,B,D]

From user D:

C_D: [C]

The reduce will now reduce lists with the same key to the intersection of the lists. For example, given A_B:[B,C] and A_B:[A,C] it will reduce them to A_B:[C] thus showing that A and B both have C as a common friend between them. The results of the final reduce will look like:

A_B:[C]

A_C:[B]

B_C:[A]

….

Here are some of the changes:

1) Your KeyValue pairs should be <String,List<String>>
2) Your input string will be of the form:
   <user profile> <user profile> … <user profile>

   That is, it contains a series of user profiles separated by spaces.
   Each <user profile> is of the form:
   <user>,<friend>,<friend>, …<friend>

That is, the string of characters before the first comma is the user's name and all comma separated strings are the user's friends. In our example, the input will be:

a,b,c b,a,c c,a,b,d d,c

Here we have four user profiles telling us that a's friends are b and c, b's friends are a and c, c's friends are a, b, and d, and d's friends are c.

3) As illustrated in the example, your Key string should use an underscore to separate the two names. With the first name being the first one that occurs alphabetically.
4) For grading purposes in your reduce, sort the list of mutual friend strings alphabetically before putting them into the mapping.

For this problem you will need to complete the following at minimum, more might be required based on your code:

1) Create a new Mapper
2) Create a new Reducer
3) Create a different server launcher that creates the associated mapper and reducer
4) Create a different client launcher that creates the associated reducer
5) Create a stand-alone class (similar to A2)

Add the following methods in your MapReduceConfiguration file:

```
	// A standalone class similar to the two classes in A2 implementing the
Facebook map reduce algorithm
	Class getStandAloneFacebookMapReduce();
	// The main class of the server facebook implementation
	Class getServerFacebookMapReduce();
	// The main class of the client facebook implementation
	Class getRemoteClientFacebookMapReduce();
```

Tips:

`Arrays.sort` and/or `Collections.sort` will come in quite useful in your mapper and reducer.

`Lists` can easily be turned into `Sets` which allow you to find the intersection.

Submission Instructions

Same as in A1 and A2.

Common Issues

1. `setBufferTracedMessages(false)` to debug why your code is failing the checks.
2. To find out how to make the console length arbitrary look at How do I increase the capacity of the Eclipse output console?
3. Open the debug perspective to view multiple different consoles for different main methods. Selecting a process makes the console show I/O for that process.

4. Make sure each client invokes the `tracewait()` call before doing its `wait()`. (Remember, you have to make the client main threads wait so they don't exit until the `quit()` method is called on the Server, which will notify the clients and print the correct traces related to quitting.) No input will be provided to the server until client 2 outputs this message. Also make sure the lookup is done after the `wait()`.
5. Don't call your match method immediately after making each slave thread — wait until all slave threads are created and added to the stack.
6. The client `quit()` method should also call `traceQuit()`, in addition to the client main thread calling `traceExit()` (after unblocking due to the `quit()` function).
7. `Locate.createRegistry(int p)` and `Locate.getRegistry(int p)` should be passed the same port number (or let them both default to 1099; either way be consistent).
8. If you're running into issues with Checkstyle, think about how you can use inheritance to separate out the data types that are needed in A3 with the ones needed in A2 for your model.